

PROJECT DOCUMENTATION

MACVEL School Management System

Smart. Secure. Connected.

Multi-Tenant SaaS School Management Platform

Document Version 1.0 · 2026-10-06

Prepared by MACVEL Development Team

Table of Contents

- 1. 01 Project Overview
- 2. 02 Requirements
- 3. 03 UI / UX
- 4. 04 Technical Documentation
- 5. 05 API Documentation
- 6. 06 Database
- 7. 07 Testing
- 8. 08 Deployment
- 9. 09 Development Log
- 10. 10 Release Notes

01 Project Overview

MACVEL School Management System

1. Project Purpose

MACVEL School Management System is a multi-tenant SaaS platform that digitises the day-to-day operations of schools. It connects every stakeholder of a school — Super Admin, School Admin, Teachers, Students and Class Monitors — through role-specific portals in a single ecosystem.

Tagline: *Smart. Secure. Connected.*

The application combines:

- Secure role-based authentication (Email / Student ID / Class Code login)
- Multi-school (multi-tenant) data isolation
- Class, Student and Teacher management
- Homework assignment, submission and grading
- Exams, Exam Schedules, Marks and Report Cards
- Weekly lesson planning (classwork / homework diary)
- Timetable grid (subject × day)
- News, Circulars, Gallery, Albums, Videos and Messages
- Academic calendar and school contacts
- File uploads (PDF / images) with Cloudinary storage
- Custom portal branding (logo, headings for web and mobile)

2. Project Objective

The main objective of MACVEL is to replace registers, notice boards and scattered spreadsheets with one secure digital system that helps schools:

- Manage multiple schools from a single Super Admin console
- Give each school a fully isolated data workspace (tenant isolation)
- Let admins manage classes, students, teachers and subjects quickly
- Let teachers publish homework, circulars and lesson plans instantly
- Let students view homework, exams, marks, timetable and report cards anywhere
- Let class monitors (class-code login) maintain their own class records
- Publish school news and circulars to everyone in real time
- Keep a secure, auditable record of academic performance

3. Users & Roles

#	Role	Access Level	Portal
1	Super Admin	All schools (tenants), platform stats, branding	Web (Admin portal)
2	School Admin	Single school — classes, students, teachers, content	Web (Admin portal)
3	Teacher	Assigned classes — homework, circulars, submissions	Web / Mobile
4	Student	Own data — homework, exams, marks, timetable, report cards	Web / Mobile

#	Role	Access Level	Portal
5	Class Monitor (Class Login)	One class via class code — class records, bulk student import	Web / Mobile

4. Functional Modules

#	Module	Description
1	Authentication & Profiles	Register, login (email or student ID), class-code login, JWT sessions, password change, profile update
2	Super Admin Console	School (tenant) CRUD, subscription/status control, admins per school, platform statistics
3	School Admin Portal	Dashboard, classes, students, teachers, subjects, contacts, exams, report cards, albums, calendar
4	Teacher Portal	Dashboard, my classes, students, homework + grading, circulars
5	Student Portal	Dashboard, homework submission, exams, marks, timetable, profile, report cards, news
6	Class Controller	Class dashboard, student CRUD with bulk import, homework, exams, schedules, subjects
7	Content & Communication	News, circulars, gallery, albums, videos, messages, posts
8	Academics	Exams, exam schedules, marks, report cards (manual + PDF/image upload), weekly lessons
9	Timetable	Weekly subject × day grid with classwork/homework cells, publish workflow
10	Calendar & Contacts	Academic calendar events per role, school contact directory
11	Media & Files	Multer uploads with Cloudinary storage (local <code>./uploads</code> fallback)
12	Portal Branding	Logo, heading and mobile branding customisation (public GET before login)

5. Supported Platforms

- **Web Browsers** — Chrome, Edge, Firefox, Safari (responsive layout), deployed on Vercel
- **Android** — Capacitor native wrapper (`mobile/` project)
- **iOS** — Capacitor native wrapper (`mobile/` project)
- **Backend API** — REST API accessible from any HTTP client (port 3000)

6. Document Index

Section	Document
01	Project Overview
02	Requirements
03	UI / UX
04	Technical Documentation
05	API Documentation
06	Database
07	Testing
08	Deployment
09	Development Log

Section	Document
10	Release Notes

Document Version: 1.0
Prepared by: MACVEL Development Team
Last Updated: 2026-10-06

02 Requirements

MACVEL School Management System

1. Functional Requirements

1.1 Authentication & Profiles

- User registration and login with **email OR student ID** (`POST /api/auth/login`)
- **Class-code login** for class monitors (`POST /api/auth/class-login`)
- JWT session persistence with role and tenant claims
- Password change (`PUT /api/auth/password`) and logout (`POST /api/auth/logout`)
- Profile view / update for every role (`GET|PUT /api/profile`)
- Bcrypt password hashing (10 salt rounds) and account-lock fields for admins

1.2 Multi-Tenancy & Super Admin

- Create, update, suspend and soft-delete schools (tenants)
- Unique school `code` and `domainSlug` per tenant
- Subscription plans (FREE / BASIC / PREMIUM / ENTERPRISE) and user limits
- Per-school admin management (create, edit, reset password, delete)
- Platform statistics, system config and DB latency monitoring
- Super-admin-only cleanup utility (stats / execute / verify)

1.3 School Admin Portal

- Dashboard with school metrics
- Class management: create/update/delete, auto-generated class code, class password reset
- Student management: CRUD, next-ID generation, pagination and search
- Teacher management: CRUD, available-teacher listing
- Subject and contact directory CRUD
- Exam content management (create / publish / delete with file URL)
- Report card management: create, publish, bulk publish, PDF/image upload
- Album management (photos + external links), calendar events
- Portal branding: web logo/heading and mobile logo/heading

1.4 Teacher Portal

- Dashboard, list of assigned classes and students
- Homework: create, edit, delete, publish; view submissions; grade with marks + feedback
- Circulars: create and list for the school

1.5 Student Portal

- Dashboard, timetable (published only), homework list with submission (file upload)
- Exams, marks and profile
- Report cards (view + parent acknowledgment)
- News list and weekly lessons (by date)

1.6 Class Controller (Class Monitor)

- Class dashboard with class code display
- Student CRUD with **next student ID** and **bulk import** (CSV/XLSX template + upload)
- Student password reset
- Homework CRUD with homework file upload/delete
- Exams and exam schedules (view), subjects CRUD
- Class timetable authoring (subject × Mon–Sat grid, classwork/homework cells) with publish

1.7 Content & Communication

- News and Circulars: CRUD, image/attachment upload, visibility (all / teachers / specific class), publish flags
- Gallery and Albums: CRUD, album links, category listing
- Videos: CRUD with category, visibility and publish flags
- Messages: inbox listing, send, delete with read receipts
- Posts: create/edit/delete own posts, post statistics

1.8 Media & File Uploads

- Single and multiple file uploads (`/api/files/upload` , `/upload-multiple`)
- Dedicated uploads: homework files, exam files, news images, report card PDFs
- **Cloudinary** primary storage; automatic local `./uploads` fallback when unconfigured
- Public static serving of `/uploads` with 1-year cache headers

2. Non-Functional Requirements

#	Requirement	Implementation
1	Security	Helmet headers, CORS allow-list, bcrypt hashing, JWT, RBAC middleware, tenant isolation middleware
2	Data Isolation	Every query scoped by <code>tenantId</code> / <code>schoolId</code> (<code>enforceSchoolIsolation</code>)
3	Availability	Health check endpoint (<code>/health</code>), graceful DB connect/disconnect handling
4	Performance	In-memory cache utilities, DB indexes on tenant + foreign keys, pagination on list APIs
5	Validation	express-validator rules on protected routes; Mongoose schema validation
6	Compatibility	Responsive web (Vite build), Android + iOS via Capacitor
7	Limits	10 MB JSON body limit, configurable <code>MAX_FILE_SIZE</code> (default 10 MB)
8	Extensibility	Modular route/controller/model structure, feature flags in tenant settings

3. Core Modules Summary

Module Name	Purpose	Key Functions	Status
Auth & Profile	Identity & sessions	Register, Login (email/ID/class code), password change, profile	Completed
Super Admin	Platform oversight	School CRUD, stats, admins, branding, cleanup	Completed
Admin Portal	School management	Classes, students, teachers, subjects, contacts, exams	Completed
Teacher Portal	Classroom management	Homework CRUD + grading, circulars, class/student lists	Completed

Module Name	Purpose	Key Functions	Status
Student Portal	Learning access	Homework submit, exams, marks, timetable, report cards	Completed
Class Controller	Class self-service	Student bulk import, homework, exams, subjects, timetable	Completed
Academics	Assessment	Exams, schedules, marks, report cards (manual + file), weekly lessons	Completed
Content	Communication	News, circulars, gallery, albums, videos, messages, posts	Completed
Timetable	Weekly plan	Subject × day grid, publish/unpublish, CSV sample	Completed
Calendar & Contacts	School events	Role-based calendar events, contact directory	Completed
Media Storage	File pipeline	Multer → Cloudinary (fallback: local disk), delete APIs	Completed
Portal Branding	White-label login	Web/mobile logo + headings, public GET before login	Completed

Document Version: 1.0
Last Updated: 2026-10-06

03 UI / UX

MACVEL School Management System

1. Design Framework

Aspect	Choice
UI Library	Ionic 7 (React) components — IonApp, IonSplitPane, IonRouterOutlet, IonPage
Icons	Ionicons 7
Styling	Global <code>theme.css</code> + per-screen <code>.css</code> files, CSS variables, gradient cards
Routing	react-router-dom v5 via <code>@ionic/react-router</code>
State	<code>AuthContext</code> (<code>AuthProvider</code> / <code>useAuth</code>) for session, role and loading states
Branding	<code>useSchoolBranding</code> hook — school logo/heading pulled from portal branding API

2. Navigation Model

- **Side menu (`IonSplitPane`)** — shown after login, differs per role:
 - `AdminMenu` for admin / super-admin roles
 - `TeacherMenu` for TEACHER role
 - `StudentMenu` for STUDENT role
- **Route guards** — unauthenticated users are redirected to `/login` (or `/student/login`); authenticated users land on their role home (`/admin` , `/teacher` , `/student`)
- **Loading state** — a full-screen loading indicator is shown while the auth context initialises (prevents blank-screen flashes)
- **Role detection** — normalised Title-Case role strings (`Student` , `Teacher` , `School Admin` , `Super Admin`) drive menu and route access

3. Screen Inventory

3.1 Authentication Screens

Screen	Route	Purpose
LoginScreen	<code>/login</code>	Admin / Teacher / Employee login with branded header
StudentLoginScreen	<code>/student/login</code>	Student login
RegisterScreen	<code>/register</code>	New account registration

3.2 Admin Screens (`frontend/src/screens/admin/`)

Screen	Purpose
AdminDashboardScreen	School metrics dashboard
AdminStudentsScreen + Create/EditStudentScreen	Student list & forms
AdminTeachersScreen + Create/EditTeacherScreen	Teacher list & forms
AdminClassesScreen + Create/EditClassScreen, ClassDetailScreen	Class management

Screen	Purpose
AdminNewsScreen, AdminCircularsScreen + CreateCircularScreen	Content publishing
CalendarScreen (admin)	Academic calendar events
AdminExamsScreen	Exam management
ReportCardsScreen, ReportCardViewScreen, StudentReportCardSearchScreen	Report cards
AdminAlbumsScreen + Create/EditAlbumScreen	Photo albums
AdminSettingsScreen	Settings

3.3 Super Admin Screens (`frontend/src/screens/superadmin/`)

DashboardScreen, SchoolsListScreen, SchoolDetailScreen, CreateSchoolScreen, PortalBrandingScreen

3.4 Teacher Screens (`frontend/src/screens/teacher/`)

TeacherDashboardScreen, TeacherClassesScreen, TeacherAttendanceScreen, TeacherHomeworkScreen, TeacherStudentsScreen

3.5 Student Screens (`mobile/student/`)

StudentDashboardScreen, StudentHomeworkScreen, StudentExamsScreen, StudentExamDetailScreen, StudentTimetableScreen, StudentMarksScreen, StudentProfileScreen, ReportCardsScreen, NewsListScreen

3.6 Class Controller Screens (`frontend/src/screens/classcontroller/`)

ClassStudentsListScreen, ClassHomeworkList/Detail/EditScreen, ClassExamSchedulesListScreen, ClassExamDetailScreen, ClassNewsList/DetailScreen, ClassProfileScreen, ClassTimetableScreen

3.7 Shared Screens (`frontend/src/screens/`)

NewsScreen, CircularsScreen, GalleryScreen, AlbumsScreen, VideosScreen, MessagesScreen, CalendarScreen, ContactsScreen, SettingsScreen + shared `TimetableGridTable`

4. UX Principles Applied

1. **Role-first design** — every user sees only the menu and screens relevant to their role; no dead-end navigation
2. **Consistent visual language** — gradient cards, emoji/ Ionicon cues, uniform list rows and detail pages across modules
3. **Feedback & safety** — confirmation dialogs before destructive actions (delete), toast/alert feedback on success and error, disabled states while saving
4. **Responsive layout** — Ionic responsive components; split-pane collapses to a hamburger menu on small screens
5. **Fast, forgiving forms** — inline validation messages, next-ID auto-fill for students, bulk import template download
6. **Brand customisation** — school logo and headings appear on login and dashboards via portal branding (works before login)
7. **Empty & loading states** — loading guards during auth init; empty-state messages when lists have no data

5. UI Screen Flow (Typical)

Login → Role detection → Role side menu + Home dashboard

└ Admin: Classes → Students / Teachers / Exams / Report cards / Content

└ Teacher: My classes → Homework → Submissions → Circulars

└ Student: Dashboard → Homework / Exams / Marks / Timetable / Report cards

└ Class monitor: Class dashboard → Students (bulk import) / Homework / Timetable

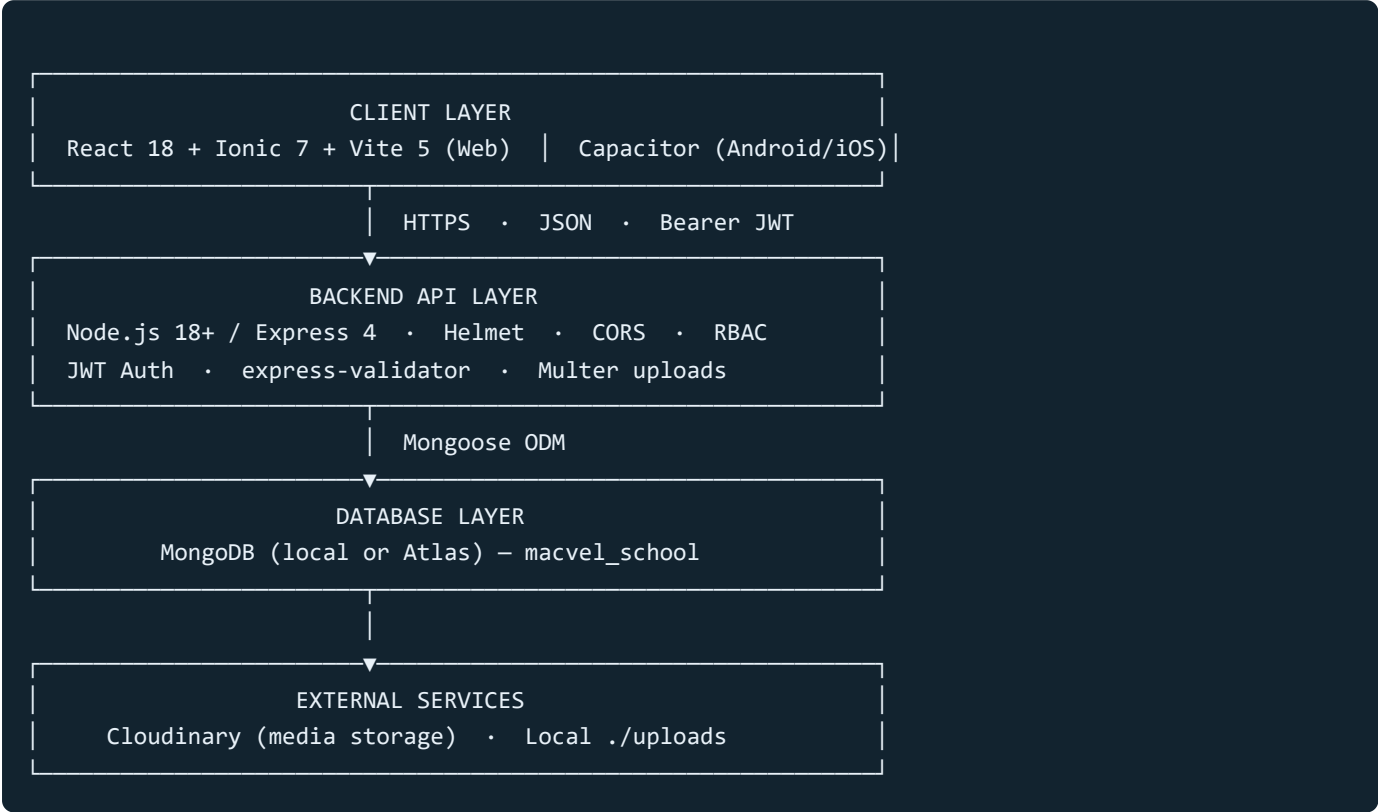
Document Version: 1.0

Last Updated: 2026-10-06

04 Technical Documentation

MACVEL School Management System

1. System Architecture



2. Client Layer

Technology: React 18 + Ionic 7 + Vite 5 (JavaScript/JSX)

Responsibilities:

- Cross-platform UI rendering (web + hybrid mobile)
- Role-based routing and guard logic (`AuthContext`)
- REST API consumption via **axios** (`src/services/api.js`)
- Forms, lists, dashboards, timetable grid rendering
- School branding application (`useSchoolBranding`)

Key directories: `frontend/src/screens/` (admin, teacher, superadmin, classcontroller, shared),
`frontend/src/components/`, `frontend/src/contexts/`, `frontend/src/services/`

3. Mobile Layer

Technology: Capacitor 8 (`@capacitor/core`, `@capacitor/android`, `@capacitor/ios`)

- Wraps the built web assets (`webDir: build`) into native Android/iOS shells (`mobile/android`, `mobile/ios`)
- App ID: `com.school.app` · App Name: `School`
- Shared React/Ionic codebase with the web client; student screens live in `mobile/student/`

4. Backend API Layer

Technology: Node.js 18+ / Express 4 (`backend/src/server.js`)

Middleware pipeline (in order):

#	Middleware	Purpose
1	Helmet	Security headers (CSP relaxed for API use)
2	CORS	Reflects request origin, credentials allowed, preflight 204 handling; allowed headers include <code>Authorization</code> , <code>x-tenant-id</code>
3	Body parsers	<code>express.json</code> / <code>urlencoded</code> with 10 MB limit
4	Static <code>/uploads</code>	Public file serving with 1-year cache
5	<code>connectDB()</code>	MongoDB connection at startup (exits on failure)
6	Public routes	<code>/health</code> , <code>/api/super-admin</code> , <code>/api/auth</code> , <code>/api/portal-branding</code>
7	<code>authenticate</code>	JWT verification for all remaining routes
8	<code>verifyRole</code> / <code>enforceSchoolIsolation</code>	RBAC + tenant scoping (per route)
9	404 handler	JSON <code>Endpoint not found</code>
10	<code>errorHandler</code>	Centralised JSON error responses

Structure:

```
backend/src/
├─ server.js           # App wiring, middleware, route mounts
├─ config/             # db.js (Mongoose), cloudinary.js
├─ controllers/        # Business logic (23 controllers)
├─ routes/             # 32 route files (router definitions)
├─ models/             # 35 Mongoose models
├─ middleware/         # auth, rbac, tenant, errorHandler, fileUpload, maintenanceGuard
├─ services/           # storageService (Cloudinary/local abstraction)
└─ utils/              # idGenerator, cache, adminCascadeDelete, cleanup scripts
```

5. Database Layer

DBMS: MongoDB via **Mongoose 9** ODM

Default URI: `mongodb://localhost:27017/macvel_school` (overridable with `MONGODB_URI` , supports Atlas)

Connection features: 5 s server-selection timeout, 45 s socket timeout, connection reuse, error/disconnect event handling, graceful SIGINT shutdown

Schema design: 35 collections; all tenant-scoped documents carry `tenantId` or `schoolId` with compound indexes

6. External Cloud Services

- **Cloudinary** — primary media asset storage (images, PDFs, exam/report files) configured via `CLOUDINARY_URL`

- **Local disk fallback** — `backend/uploads/` (`images` , `news` , `circulars` , `homework` , `exam` , `reportcard` , `audio` , `videos` , `documents`) when Cloudinary is unset
- **Vercel** — production hosting for the web frontend

7. Authentication & Authorization

Token: JWT (HS256), expiry `JWT_EXPIRES_IN` (default 7d), sent as `Authorization: Bearer <token>`

Claims: `id` , `email` , `role` , `tenantId` / `schoolId` , `classId` (class login)

Role	Scope
Super Admin	All tenants
School Admin	One school
Teacher	Assigned classes
Student	Own records only
Class login	Single class (class code + password)

Password security: bcrypt hashing (10 rounds, configurable `BCRYPT_SALT_ROUNDS`), `select: false` on password fields, login-attempt/lock fields on Admin accounts.

8. Request Lifecycle (Example: create homework)

1. Teacher logs in → `POST /api/auth/login` → JWT issued
2. Client calls `POST /api/homework` with Bearer token
3. `authenticate` decodes JWT → `req.user` populated
4. Route-level `verifyRole('Teacher', ...)` validates role
5. Controller validates body (express-validator / Mongoose)
6. Document saved with `schoolId` / `tenantId` of the requester
7. JSON response `{ success: true, ... }`; errors funnel to `errorHandler`

Document Version: 1.0

Last Updated: 2026-10-06

05 API Documentation

MACVEL School Management System

1. API Overview

Item	Value
Base URL (dev)	http://localhost:3000/api
Health check	GET /health (outside /api)
Format	JSON (UTF-8)
Authentication	Authorization: Bearer <JWT>
Optional header	x-tenant-id — tenant context for scoped requests
Success shape	{ "success": true, ... }
Error shape	{ "success": false, "error": { "message", "path", "method" } }

Public routes (no token): /health , /api/auth/* (login, register, class-login), /api/super-admin/* , /api/superadmin/* , GET /api/portal-branding . All other routes pass through the global authenticate middleware.

2. Authentication — /api/auth

Method	Endpoint	Auth	Description
POST	/login	No	Login with email, username or student ID + password
POST	/register	No	Register a new user
POST	/class-login	No	Login with class code + class password
GET	/me	Yes	Current user profile from token
PUT	/password	Yes	Change password
POST	/logout	Yes	Client-side logout

Sample request:

```
POST /api/auth/login
{
  "usernameOrEmailOrId": "admin@school.com",
  "password": "password123"
}
```

Sample response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": { "id": "...", "name": "...", "role": "School Admin", "schoolId": "..." }
}
```

3. Super Admin — `/api/super-admin` (alias `/api/superadmin`)

Method	Endpoint	Description
GET	<code>/stats</code>	Platform-wide statistics
POST	<code>/schools</code>	Create a school (tenant)
GET	<code>/schools</code>	List all schools
GET	<code>/schools/:id</code>	School details
PUT	<code>/schools/:id</code>	Update school
PATCH	<code>/schools/:id/status</code>	Activate / suspend school
DELETE	<code>/schools/:id</code>	Soft-delete school
GET	<code>/schools/:schoolId/admins</code>	List admins of a school
PUT	<code>/schools/:schoolId/admins/:adminId</code>	Update school admin
DELETE	<code>/schools/:schoolId/admins/:adminId</code>	Remove school admin
POST	<code>/schools/:schoolId/admins/:adminId/reset-password</code>	Reset admin password

4. Portal Branding — `/api/portal-branding`

Method	Endpoint	Auth	Description
GET	<code>/</code>	No	Branding for login page (public)
PUT	<code>/</code>	Super Admin	Update heading / sub-heading
PUT	<code>/logo</code>	Super Admin	Upload web logo
DELETE	<code>/logo</code>	Super Admin	Remove web logo
PUT	<code>/mobile-logo</code>	Super Admin	Upload mobile logo
DELETE	<code>/mobile-logo</code>	Super Admin	Remove mobile logo
POST	<code>/reset</code>	Super Admin	Reset branding to defaults
DELETE	<code>/mobile</code>	Super Admin	Clear mobile branding

5. School Admin — `/api/admin`

Method	Endpoint	Description
GET	<code>/dashboard</code>	Admin dashboard metrics
GET	<code>/classes · /classes/:id · /classes/:id/dashboard</code>	Class list / details / dashboard

Method	Endpoint	Description
POST	<code>/classes</code>	Create class (auto class code)
PUT · DELETE	<code>/classes/:id</code>	Update / delete class
POST	<code>/classes/:id/reset-password</code>	Reset class login password
POST	<code>/reset-class-code-counter</code>	Reset class code sequence (password guarded)
GET	<code>/students</code> · <code>/students/:id</code> · <code>/students/next-id</code>	Student list / details / next ID
POST · PUT · DELETE	<code>/students</code> · <code>/students/:id</code>	Student CRUD
GET	<code>/teachers</code> · <code>/teachers/:id</code> · <code>/teachers/available</code>	Teacher list / details / unassigned
POST · PUT · DELETE	<code>/teachers</code> · <code>/teachers/:id</code>	Teacher CRUD
GET · POST · PUT · DELETE	<code>/subjects</code> · <code>/subjects/:id</code>	Subject CRUD
GET · POST · PUT · DELETE	<code>/contacts</code> · <code>/contacts/:id</code>	Contact directory CRUD
GET	<code>/exams</code>	Exam content list
POST · PUT · DELETE	<code>/content/exams</code> · <code>/content/exams/:id</code>	Exam content CRUD

6. Teacher — `/api/teacher`

Method	Endpoint	Description
GET	<code>/dashboard</code>	Teacher dashboard
GET	<code>/classes</code>	Assigned classes
GET	<code>/students</code>	Students of assigned classes
GET · POST · PUT · DELETE	<code>/homework</code> · <code>/homework/:id</code>	Homework CRUD
GET	<code>/homework/:id/submissions</code>	Submissions for a homework
PUT	<code>/homework/:id/submissions/:studentId</code>	Grade a submission (marks + feedback)
GET · POST	<code>/circulars</code>	School circulars

7. Student — `/api/student`

Method	Endpoint	Description
GET	<code>/dashboard</code>	Student dashboard
GET	<code>/timetable</code>	Published timetable
GET	<code>/homework</code>	Homework list with submission status
POST	<code>/homework/:id/submit</code>	Submit homework (file URL)
GET	<code>/exams</code>	Exam list
GET	<code>/marks</code>	Marks list

Method	Endpoint	Description
GET	/profile	Student profile

8. Class Controller — /api/class-controller

Method	Endpoint	Description
GET	/dashboard	Class dashboard (class code, counts)
GET · POST · PUT · DELETE	/students · /students/:id	Class student CRUD
GET	/students/next-id	Next auto student ID
PUT	/students/:id/reset-password	Reset student password
GET	/students/bulk-import/template	Download import template
POST	/students/bulk-import	Bulk import students (CSV/XLSX)
GET · POST · PUT · DELETE	/homework · /homework/:id	Class homework CRUD
POST · DELETE	/upload-homework-files · /homework-files/:filename	Homework file upload / delete
GET	/exams · /exams/:id	Class exams
GET	/exam-schedules · /exam-schedules/:id	Exam schedules
GET · POST · PUT · DELETE	/subjects · /subjects/:id	Class subjects CRUD

9. Homework — /api/homework

Method	Endpoint	Description
GET · POST · PUT · DELETE	/ · /:id	Homework CRUD (published only by default)
POST	/:id/submit	Student submission
POST	/:id/check	Teacher grading action

10. Exams — /api/exams

Method	Endpoint	Description
GET · POST · PUT · DELETE	/ · /:id	Exam CRUD
GET	/schedule/:classId	Schedule for a class
POST	/:id/schedule	Add exam schedule
PUT · DELETE	/schedule/:id	Update / delete schedule

11. Timetable — `/api/timetable`

Method	Endpoint	Description
GET	<code>/template</code>	Timetable template
GET	<code>/admin/all</code> · <code>/admin/:id</code>	Admin list / detail
GET	<code>/</code>	Timetable for current class/school
POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Timetable CRUD
PATCH	<code>/:id/publish</code>	Publish / unpublish

12. Content APIs

News — `/api/news`

Method	Endpoint	Description
GET · POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	News CRUD

Circulars — `/api/circulars`

Method	Endpoint	Description
GET · POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Circular CRUD

Gallery — `/api/gallery`

Method	Endpoint	Description
GET · POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Gallery item CRUD

Videos — `/api/videos`

Method	Endpoint	Description
GET	<code>/</code> · <code>/:id</code> · <code>/categories/list</code>	Video list / detail / categories
GET	<code>/admin/all</code>	Admin video list
POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Video CRUD

Albums — `/api/albums`

Method	Endpoint	Description
GET	<code>/</code> · <code>/:id</code> · <code>/categories/list</code>	Album list / detail / categories
GET	<code>/admin/all</code> · <code>/admin/:id</code>	Admin album list / detail
POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Album CRUD
POST · DELETE	<code>/:id/links</code> · <code>/:id/links/:linkIndex</code>	External links in album

Public content — `/api/content` (read-only)

Method	Endpoint	Description
GET	<code>/news</code> · <code>/news/:id</code>	Published news

Method	Endpoint	Description
GET	<code>/circulars</code> · <code>/circulars/:id</code>	Published circulars
GET	<code>/exams</code> · <code>/exams/:id</code>	Published exams
GET	<code>/exam-schedules</code> · <code>/exam-schedules/:id</code>	Published schedules

Admin content — `/api/admin-content`

Method	Endpoint	Description
GET · POST · PUT · DELETE	<code>/news</code> · <code>/news/:id</code>	News management with visibility rules
POST	<code>/upload</code>	Generic file upload
POST	<code>/upload-exam</code>	Exam file upload (PDF/image)

Posts — `/api/posts`

Method	Endpoint	Description
GET	<code>/</code> · <code>/my-posts</code> · <code>/:id</code>	Post feeds
POST · PUT · DELETE	<code>/</code> · <code>/:id</code>	Post CRUD (own posts)

Messages — `/api/messages`

Method	Endpoint	Description
GET	<code>/</code> · <code>/:id</code>	Inbox / message detail
POST · DELETE	<code>/</code> · <code>/:id</code>	Send / delete message

13. Academics APIs

Weekly Lessons — `/api/weekly-lessons`

Method	Endpoint	Description
GET	<code>/</code>	All lessons (school scope)
GET · POST · DELETE	<code>/teacher/weekly-lessons</code> · <code>/:id</code>	Teacher lesson plans
GET	<code>/student/weekly-lessons</code> · <code>/student/weekly-lessons/by-date</code>	Student view (by date)

Report Cards — `/api/reportcards`

Method	Endpoint	Description
POST	<code>/upload</code> · <code>/bulk-upload</code>	Report card file upload (single/bulk)
POST · GET · PUT · DELETE	<code>/</code> · <code>/:id</code>	Report card CRUD
GET	<code>/school-info</code>	School details for report header
GET	<code>/by-student</code>	Filter by student
PUT	<code>/:id/publish</code>	Publish report card
PUT	<code>/class/:classId/publish-all</code>	Publish all for a class

Method	Endpoint	Description
GET	<code>/student/my-report-cards</code>	Student's own report cards
GET · PUT	<code>/student/:id</code> · <code>/student/:id/acknowledge</code>	View / parent acknowledgment

Calendar — `/api/calendar`

Method	Endpoint	Description
GET · POST · PUT · DELETE	<code>/admin/calendar-events</code> · <code>/:id</code>	Admin event CRUD
GET	<code>/student/calendar-events</code>	Student events
GET	<code>/class-controller/calendar-events</code>	Class monitor events

Contacts — `/api/contacts`

Method	Endpoint	Description
GET	<code>/</code>	School contact directory

14. Profile, Files & Misc

Mount	Method	Endpoint	Description
<code>/api/profile</code>	GET · PUT	<code>/</code>	View / update own profile
<code>/api/files</code>	POST	<code>/upload</code> · <code>/upload-multiple</code>	Upload files (multer → Cloudinary/local)
<code>/api/files</code>	DELETE	<code>/:filename</code>	Delete uploaded file
<code>/api/school-context</code>	GET	<code>/me</code>	School name + logo for dashboards
<code>/api/tenants</code>	GET · POST	<code>/</code>	Tenant list / create
<code>/api/tenants</code>	GET · PUT · DELETE	<code>/:id</code>	Tenant CRUD
<code>/api/tenants</code>	PUT · DELETE	<code>/:id/logo</code>	Tenant logo upload / remove
<code>/api/tenants</code>	GET	<code>/:id/stats</code>	Tenant statistics
<code>/api/cleanup</code>	GET · POST	<code>/stats</code> · <code>/execute</code>	Super-admin cleanup (guarded)
<code>/api/cleanup</code>	GET	<code>/verify</code>	Verify cleanup results
<code>/api/products</code>	GET · POST · PUT · DELETE	<code>/</code> · <code>/:id</code> (legacy/demo)	Product CRUD + <code>/stats</code>

15. Status Codes

Code	Meaning
200	Success
201	Resource created
204	Success, no content (OPTIONS preflight)
400	Validation error

Code	Meaning
401	Missing / invalid token
403	Role or tenant access denied
404	Endpoint or resource not found
500	Server error (via <code>errorHandler</code>)

Document Version: 1.0
Last Updated: 2026-10-06

06 Database

MACVEL School Management System

1. Database Overview

Item	Value
DBMS	MongoDB (document store)
ODM	Mongoose 9 (<code>backend/src/config/db.js</code>)
Default database name	<code>macvel_school</code>
Connection URI	<code>MONGODB_URI</code> env (local <code>mongodb://localhost:27017/macvel_school</code> or MongoDB Atlas)
Timeouts	Server selection 5000 ms · Socket 45000 ms
Unicode	MongoDB UTF-8 by default — full Tamil/name support
Total collections	35 Mongoose models

Design principles

- **Tenant isolation** — every school-scoped document carries `tenantId` or `schoolId` (ObjectId → Tenant/School) and queries are filtered by it
- **Soft delete** — `deletedAt` / `isActive` / `isPublished` flags instead of hard deletes where audit matters
- **Compound indexes** — `(tenantId, ...)` , `(schoolId, role)` , `(student, academicYear, term)` etc. for list performance
- **Password hygiene** — `select: false` on password fields; bcrypt pre-save hashing (10 salt rounds)
- **Default filters** — `pre(/^find/)` hooks auto-filter inactive users, deleted tenants, unpublished homework

2. Master Collection Summary (All 35)

#	Collection	Category	Key References	Purpose
1	<code>users</code>	Users	<code>schoolId</code> , <code>classId</code> , <code>createdBy</code>	All user accounts (students, teachers, parents) with roles
2	<code>admins</code>	Users	<code>schoolId</code> , <code>tenantId</code>	Super Admin & School Admin accounts (separate for isolation)
3	<code>teachers</code>	Users	<code>tenantId</code>	Teacher records: qualification, subjects, <code>teacherId</code>
4	<code>studentprofiles</code>	Users	<code>userId</code> , <code>schoolId</code> , <code>classId</code>	Admission no., roll no., parent details, medical notes
5	<code>tenants</code>	Tenancy	—	Schools/organizations: code, slug, plan, limits, status
6	<code>schools</code>	Tenancy	—	School profile: name, code, address, logo, subscription
7	<code>classes</code>	Academics	<code>tenantId</code> , <code>teacherId</code> , <code>createdBy</code>	Class/section with auto class code + login password

#	Collection	Category	Key References	Purpose
8	classsubjects	Academics	tenantId, classId, subjectId, teacherId	Subject-teacher mapping per class
9	subjects	Academics	tenantId	Subject master (name, code)
10	exams	Academics	classId, tenantId	Exam header: name, type, dates, file
11	examschedules	Academics	examId, classId, tenantId	Per-subject date/time/duration/room
12	marks	Academics	studentId, tenantId	Marks, percentage, grade per exam type
13	reportcards	Academics	student, schoolId, classId	Term report cards (marks grid or uploaded file)
14	timetables	Academics	tenantId, classId	Subject × Mon–Sat grid with classwork/homework
15	weeklylessons	Academics	tenantId, classId, subjectId, teacherId	Weekly lesson plans (objectives, content, status)
16	weeklylessonlogs	Academics	tenantId, classId	Daily diary log: classwork/homework text
17	homeworks	Homework	schoolId, classId, teacher	Assignments with embedded submissions
18	homeworksubmissions	Homework	homeworkId, studentId, tenantId	Standalone submission records with grade/remarks
19	news	Content	tenantId, authorId, classId	School news with priority/visibility/publish
20	circulars	Content	tenantId, authorId	Official circulars with attachments
21	classcirculars	Content	tenantId, classId, createdBy	Class-specific circulars (circular no., issue date)
22	announcements	Content	tenantId, authorId	Pinned announcements with expiry & view count
23	posts	Content	userId, tenantId	User posts feed
24	messages	Content	tenantId, senderId, classId	Internal messages with read receipts
25	mediagalleries	Media	schoolId, uploadedBy	Gallery items (image/video) with order
26	photoalbums	Media	tenantId, createdBy	Album containers with cover image
27	photos	Media	albumId, tenantId	Photos inside albums
28	albums	Media	tenantId, createdBy	Content albums with external links + visibility
29	videos	Media	tenantId, uploadedBy	Video catalogue with category/visibility/views
30	files	Media	tenantId, uploadedBy	Uploaded file registry (name, mime, size, URL)
31	calendarevents	Calendar	schoolId, classId, createdBy	Events with visibility, recurrence, timing
32	academiccalendars	Calendar	tenantId	Academic calendar entries by event type/audience
33	schoolcontacts	Misc	tenantId	Staff contact directory (department, phone, email)
34	portalbrandings	Misc	updatedBy	Web/mobile logo + heading white-label settings

#	Collection	Category	Key References	Purpose
35	products	Misc	tenantId, createdBy	Legacy/demo product catalogue with stock

3. Detailed Collection Schemas

3.1 Collection: users

Field	Type	Constraints / Default	Description
_id	ObjectId	Auto	Unique user ID
name	String	required, 2–100 chars, trim	Full name
email	String	required, unique, lowercase	Login email
password	String	required, min 6, <code>select: false</code>	Bcrypt hash (never returned)
phone	String	optional, pattern-validated	Contact number
profileImage	String	optional	Avatar URL/path
role	String	enum: Super Admin / School Admin / Teacher / Student / Parent; default <code>Student</code>	Role (auto Title-Cased)
schoolId	ObjectId → <code>schools</code>	required except Super Admin	Tenant scope
isActive	Boolean	default <code>true</code>	Soft-disable flag
dateOfBirth	Date	optional	DOB
gender	String	enum: Male / Female / Other / "	Gender
age	Number	0–150	Age
qualification	String	optional	For teachers/parents
address	Object	street, city, state, zipCode, country	Address block
studentId	String	optional, indexed	Student roll/ID number
createdBy	ObjectId → <code>users</code>	indexed	Admin/class that created the record
classId	ObjectId → <code>classes</code>	optional	Assigned class (students)
rollNumber	String	optional	Class roll number
admittedDate	Date	optional	Admission date
parentOf	[ObjectId] → <code>users</code>	optional	Parent's children links
emergencyContact	Object	name, phone, relationship	Emergency contact
createdAt / updatedAt	Date	timestamps	Audit timestamps

Indexes: `email` (unique) · `(schoolId, role)` · `(schoolId, studentId)` · `(role, isActive)`

Hooks: bcrypt pre-save hashing · `pre-find` hides inactive users by default

3.2 Collection: `classes`

Field	Type	Constraints / Default	Description
<code>name</code>	String	required, trim	Class name (e.g., "Grade 6")
<code>section</code>	String	trim	Section (A/B/...)
<code>classCode</code>	String	unique, uppercase, auto-generated	Class login code (e.g., <code>GRE001</code>)
<code>tenantId</code>	ObjectId → <code>tenants</code>	required, indexed	School scope
<code>teacherId</code>	ObjectId → <code>teachers</code>	indexed	Class teacher
<code>createdBy</code>	ObjectId → <code>users</code>	indexed	Creator (admin/teacher)
<code>password</code>	String	<code>select: false</code> , bcrypt pre-save	Class login password
<code>gradeLevel</code>	Number	1–12	Grade level
<code>roomNumber</code>	String	optional	Room
<code>capacity</code>	Number	default <code>30</code>	Max students
<code>academicYear</code>	String	optional	e.g., 2026-2027
<code>isActive</code>	Boolean	default <code>true</code>	Active flag
<code>createdAt</code> / <code>updatedAt</code>	Date	timestamps	Audit timestamps

Indexes: `(tenantId, name)` · `(tenantId, section)` · `classCode` (unique)

Virtuals: `teacher`, `studentCount`, `homeworkCount`, `examCount`

3.3 Collection: `tenants`

Field	Type	Constraints / Default	Description
<code>name</code>	String	required, trim	School name
<code>domainSlug</code>	String	required, unique, lowercase	URL slug
<code>code</code>	String	required, unique, uppercase	School code
<code>address</code> · <code>phone</code> · <code>email</code>	String	optional	Contact details
<code>schoolLogoUrl</code>	String	optional	Logo URL
<code>subscriptionPlan</code>	String	enum FREE/BASIC/PREMIUM/ENTERPRISE; default <code>FREE</code>	Plan
<code>maxUsers</code>	Number	default <code>100</code>	User limit
<code>maxStudents</code>	Number	default <code>1000</code>	Student limit
<code>status</code>	String	enum ACTIVE/SUSPENDED/INACTIVE; default <code>ACTIVE</code>	Tenant status
<code>settings</code>	Object	<code>academicYear</code> , <code>timezone</code> , <code>language</code> , <code>currency</code> , <code>dateFormat</code> , <code>features</code>	Feature flags
<code>deletedAt</code>	Date	soft delete	Deletion marker

Field	Type	Constraints / Default	Description
createdAt / updatedAt	Date	timestamps	Audit timestamps

Indexes: (domainSlug, deletedAt) · (status) · pre-find excludes soft-deleted tenants

3.4 Collection: admins

Field	Type	Constraints / Default	Description
name	String	required, 2–100 chars	Admin full name
email	String	required, unique index, lowercase	Login email
password	String	required, min 6, select: false	Bcrypt hash
phone · profileImage	String	optional	Contact / avatar
role	String	enum: Super Admin / School Admin	Admin role (Title-Case setter)
schoolId	ObjectId → schools	required for School Admin	Tenant scope
tenantId	ObjectId → tenants	optional	Alternate tenant link
isActive	Boolean	default true	Active flag
dateOfBirth · gender · address · emergencyContact	Date / String / Object	optional	Profile extras
lastLogin	Date	audit	Last login time
passwordChangedAt	Date	set on hash	Password rotation
loginAttempts	Number	default 0	Failed-attempt counter
lockUntil	Date	optional	Account lock window
createdAt / updatedAt	Date	timestamps	Audit timestamps

Indexes: email (unique) · (schoolId, role) · (role, isActive) · methods: comparePassword(), isLocked()

3.5 Collection: homeworks

Field	Type	Constraints / Default	Description
title	String	required, 2–200 chars	Homework title
subject	String	required	Subject
classGrade	String	required	Class/grade label
description	String	max 2000 chars	Instructions
givenDate	Date	required, default Date.now	Assigned on

Field	Type	Constraints / Default	Description
dueDate	Date	required	Deadline
attachments	[String]	URL or /uploads/... path	Attachment files
teacher	ObjectId → users	required	Assigning teacher
schoolId	ObjectId → schools	required	Tenant scope
classId	ObjectId → classes	optional	Target class
submissions	[Sub-document]	embedded, default []	Per-student submission
- student	ObjectId → users	required	Submitter
- submissionFile	String	valid URL	Submitted file
- status	String	enum Submitted/Pending/Checked; default Pending	Status
- submittedAt / - checkedAt	Date	auto on status change	Timestamps
- marks / - maxMarks	Number	≥ 0 / ≥ 1	Grading
- feedback	String	max 500 chars	Teacher feedback
isPublished	Boolean	default false	Students see only published
maxMarks	Number	default 100	Total marks
createdAt / updatedAt	Date	timestamps	Audit timestamps

Hooks: pre-find returns published homework only (bypass option for admins)

Methods: addSubmission(), checkSubmission(), findByClass/Teacher/Student(), findUpcoming/Overdue()

3.6 Collection: reportcards

Field	Type	Constraints / Default	Description
student	ObjectId → users	required, indexed	Student
schoolId	ObjectId → schools	required	Tenant scope
classId	ObjectId → classes	optional	Class
className · classSection	String	optional	Denormalised class info
createdBy · sentBy	ObjectId → users	optional	Audit refs

Field	Type	Constraints / Default	Description
term	String	enum: Term 1–3, Half Yearly, Annual, Unit Test 1–3; required	Exam term
academicYear	String	required, YYYY-YYYY format	Academic year
subjects	[Sub-doc]	subjectName, marksObtained, totalMarks, grade, remarks	Per-subject marks
totalPercentage	Number	required, 0–100	Overall %
overallGrade	String	required	Overall grade
rank	Number	≥ 1, default null	Class rank
totalStudents	Number	optional	Batch size
attendance	Object	present, total, percentage	Attendance summary
pdfReportUrl	String	valid URL	Generated PDF link
reportCardFileUrl	String	valid URL	Uploaded PDF/image
reportCardFileType	String	enum pdf/image/null	Uploaded file type
teacherRemarks · principalRemarks	String	max 500 chars	Remarks
parentAcknowledgment	Boolean	default false	Parent confirmation
parentSignature	String	optional	Signature value
isPublished	Boolean	default false	Visibility flag
publishedAt · issuedDate	Date	optional	Publish/issue dates
createdAt / updatedAt	Date	timestamps	Audit timestamps

Indexes: (student, academicYear, term) · (schoolId, academicYear, term) · (schoolId, isPublished) · (student, isPublished, issuedDate)

Rule: either subjects[] OR reportCardFileUrl must exist (schema validator)

3.7 Collection: timetables

Field	Type	Constraints / Default	Description
title	String	required, max 200	Timetable title
weekLabel	String	max 100, default ''	Week label
description	String	max 1000, default ''	Notes
tenantId	ObjectId → tenants	required, indexed	School scope
classId	ObjectId → classes	required, indexed	Class scope
createdBy	Mixed	optional	Creator id

Field	Type	Constraints / Default	Description
createdByRole	String	enum ADMIN/CLASS/TEACHER/OTHER; default CLASS	Author role
rows	[Row]	default []	Grid rows
– subject	String	required, max 80	Row subject
– Monday...Saturday	Day cell	{ classwork: String, homework: String }	Per-day cells
isPublished	Boolean	default false, indexed	Student visibility
isActive	Boolean	default true	Active flag
createdAt / updatedAt	Date	timestamps	Audit timestamps

Indexes: (tenantId, classId, isPublished) · (tenantId, updatedAt) · collection name: timetables

4. Remaining Collections — Key Fields Catalog

#	Collection	Key Fields
1	teachers	email, password, name, phone, profileImage, tenantId, teacherId, qualification, experienceYears, specialization[], dateOfBirth, gender, address, isActive, loginAttempts, lockUntil
2	studentprofiles	userId, schoolId, classId, admissionNumber, rollNumber, parentName, parentPhone, bloodGroup, medicalHistory
3	schools	schoolName, schoolCode, address, contactEmail, contactPhone, schoolLogoUrl, status, subscriptionExpiry, settings
4	classsubjects	tenantId, classId, subjectId, teacherId
5	subjects	name, code, description, tenantId, isActive
6	exams	name, type, classId, tenantId, startDate, endDate, isPublished, description, fileUrl, academicYear
7	examschedules	title, subject, examId, classId, tenantId, date, startTime, endTime, duration, roomNo, isPublished, fileUrl
8	marks	studentId, subject, subjectId, tenantId, marksObtained, totalMarks, percentage, grade, examType, examDate, remarks, isPublished
9	weeklylessons	tenantId, classId, subjectId, teacherId, weekStartDate, title, objectives, content, status
10	weeklylessonlogs	tenantId, classId, subject, lessonDate, classworkText, homeworkText, createdBy, updatedBy
11	homeworksubmissions	homeworkId, studentId, tenantId, submissionText, attachmentUrl, status, grade, remarks, gradedBy, submittedAt
12	news	title, content, type, tenantId, authorId, isPublished, publishedAt, imageUrl, attachmentUrl, priority, visibility, classId
13	circulars	title, content, tenantId, authorId, isPublished, publishedAt, imageUrl, attachmentUrl, visibility, classId, expiryDate
14	classcirculars	tenantId, classId, title, content, circularNo, createdBy, isPublished, issueDate

#	Collection	Key Fields
15	announcements	title, content, type, tenantId, authorId, targetAudience, isPublished, publishedAt, expiryDate, imageUrl, priority, isPinned, viewCount
16	posts	title, content, userId, tenantId, isPublished
17	messages	tenantId, senderId, recipientType, recipientId, classId, subject, message, attachmentUrl, isImportant, isRead, readAt
18	mediagalleries	title, category, mediaType, url, description, eventDate, schoolId, uploadedBy, isPublished, thumbnailUrl, order
19	photoalbums	tenantId, title, description, coverImageUrl, eventDate, isPublished, createdBy, isActive
20	photos	albumId, tenantId, imageUrl, caption, sortOrder, isActive
21	albums	title, description, tenantId, createdBy, visibility, isPublished, isActive, category
22	videos	title, description, videoUrl, thumbnailUrl, category, duration, tenantId, uploadedBy, visibility, isPublished, isActive, views
23	files	tenantId, fileName, originalName, mimeType, size, url, uploadedBy, isActive
24	calendarevents	title, description, date, endDate, eventType, color, schoolId, visibility, classId, createdBy, isPublished, isRecurring, location, startTime, endTime
25	academiccalendars	tenantId, title, description, eventType, startDate, endDate, isRecurring, recurringPattern, targetAudience, color, isActive
26	schoolcontacts	tenantId, department, name, designation, phone, email, isActive
27	portalbrandings	key, logoUrl, showLogo, heading, subHeading, mobileLogoUrl, mobileHeading, mobileSubHeading, showMobileLogo, updatedBy
28	products	tenantId, name, description, sku, price, stockQuantity, category, metadata, createdBy, isActive, deletedAt

5. Entity Relationship Overview

```

tenants (schools)
├── classes ───┬── users (students, classId)
│               ├── timetables
│               ├── homeworks ─ embedded submissions
│               ├── exams ─ examschedules
│               ├── classsubjects ─ subjects
│               └── classcirculars / weeklylessons / weeklylessonlogs
├── users (teachers, admins)
├── marks / reportcards (student + schoolId + term)
├── news / circulars / announcements / albums / videos / calendar
└── schoolcontacts / portalbrandings

```

Cardinality notes

- One tenant → many classes, users, content documents (1:N via `tenantId` / `schoolId`)
- One class → many students, homeworks, exams, timetables (1:N via `classId`)
- One homework → embedded submissions array (1:1 embedded, 1:N alternative in `homeworksubmissions`)

- One student → many marks, report cards, homework submissions (1:N via `student` / `studentId`)
-

Document Version: 1.0

Last Updated: 2026-10-06

07 Testing

MACVEL School Management System

1. Testing Approach

Testing in the current release is **manual, script-driven API testing** combined with lint checks. The repository does not yet include an automated unit-test framework (`npm test` is a placeholder that exits 0). Dedicated Node test scripts exercise real endpoints against a running server and report pass/fail in the console.

2. Test Environment

Item	Value
Runtime	Node.js 18+ (tested on v22)
Server	<code>node src/server.js</code> on <code>http://localhost:3000</code>
Database	MongoDB (local / Atlas) — <code>macvel_school</code>
Test client	axios (Node scripts)
Frontend lint	ESLint (<code>npm run lint</code> in <code>frontend/</code>)
API base URL	<code>http://localhost:3000/api</code> (Android emulator: <code>http://10.0.2.2:3000/api</code>)

3. Test Scripts

#	Script	Purpose	Run
1	<code>test-teacher-endpoint.js</code>	Verifies teacher <code>GET /api/teacher/my-class</code> (404-fix regression) — login → authorised call → status assert	<code>node test-teacher-endpoint.js</code>
2	<code>test-weekly-lessons.js</code>	Full CRUD cycle for weekly lessons with TEACHER + STUDENT tokens	<code>node test-weekly-lessons.js</code>
3	<code>backend/scripts/test-cloudinary.js</code>	Cloudinary credentials & upload connectivity check	<code>node scripts/test-cloudinary.js</code>
4	<code>backend/scripts/test-upload-pipeline.js</code>	Spins a test Express app to verify <code>uploadSingle('file')</code> → URL resolution	<code>node scripts/test-upload-pipeline.js</code>
5	<code>backend/scripts/make-student-import-samples.js</code>	Generates sample CSV/XLSX files for bulk-import testing	<code>node scripts/make-student-import-samples.js</code>
6	<code>backend/tmp_verify.js</code> , <code>tmp_admin_timetable_check.js</code> , <code>tmp_exam_upload_check.js</code>	Ad-hoc verification of timetable & exam-upload fixes	<code>node tmp_*.js</code>
7	Frontend ESLint	Static analysis of <code>src</code> JSX/JS	<code>npm run lint</code>

4. Manual Test Cases

4.1 Authentication

ID	Test Case	Expected Result	Status
AUTH-01	Login with valid email + password	200, JWT + user object returned	Pass
AUTH-02	Login with student ID (e.g., <code>STU0001</code>)	200, student token issued	Pass
AUTH-03	Login with wrong password	401, generic error message	Pass
AUTH-04	Class-code login (<code>classCode</code> + password)	200, class-scoped token	Pass
AUTH-05	Access protected route without token	401 <code>Unauthorized</code>	Pass
AUTH-06	Change password via <code>PUT /api/auth/password</code>	200, new password usable next login	Pass
AUTH-07	Tampered / expired JWT	401, token rejected	Pass

4.2 Role-Based Access Control

ID	Test Case	Expected Result	Status
RBAC-01	Student calls <code>/api/admin/classes</code>	403 Forbidden	Pass
RBAC-02	Teacher accesses only assigned class data	Other classes excluded	Pass
RBAC-03	School Admin of school A queries school B data	Empty/denied — tenant isolation holds	Pass
RBAC-04	Super Admin manages all tenants	Full access to <code>/api/super-admin/*</code>	Pass

4.3 Core CRUD

ID	Test Case	Expected Result	Status
CRUD-01	Create class → auto class code generated	Unique code (e.g., <code>GRE001</code>) returned	Pass
CRUD-02	Create student → next student ID auto-filled	Sequential ID returned by <code>/students/next-id</code>	Pass
CRUD-03	Edit / delete teacher	200, list reflects change	Pass
CRUD-04	Homework create → publish → student sees it	Published-only filter works	Pass
CRUD-05	Student submits homework file	Submission status becomes <code>Submitted</code>	Pass
CRUD-06	Teacher grades submission	Marks + feedback stored, status <code>Checked</code>	Pass
CRUD-07	Report card create → publish → student view	Visible only after publish	Pass

4.4 Content & Media

ID	Test Case	Expected Result	Status
MEDIA-01	Upload news image (multer → Cloudinary)	Secure URL returned, news shows image	Pass
MEDIA-02	Upload exam PDF via <code>/api/admin-content/upload-exam</code>	PDF stored and linked	Pass
MEDIA-03	Cloudinary unset → local fallback	File written to <code>./uploads</code> , URL resolves	Pass

ID	Test Case	Expected Result	Status
MEDIA-04	Delete uploaded file	File removed from storage	Pass

5. Defects Found & Fixed (Regression Log)

Date	Defect	Fix
2026-08-09	Content routes failed role checks (case mismatch)	Title-Case role comparisons (<code>Student</code> , <code>Teacher</code>)
2026-08-09	Profile endpoint queried non-existent <code>tenantId</code>	Switched to <code>schoolId</code>
2026-08-26	Deep links 404 on Vercel after refresh	<code>vercel.json</code> SPA rewrites
2026-09-08	Admin delete/edit of unpublished albums/videos/homework → 404	Query filters corrected
2026-09-08	Homework visible in admin side (business rule)	Homework feature removed from admin side
2026-09-30	Media storage dependency	Cloudinary pipeline with local fallback

6. Future Testing Scope

- 1. Unit tests — Jest + Supertest for controllers/middleware
- 2. API integration suite — Newman collection exported from the endpoint tables
- 3. E2E tests — Cypress for login → role dashboard journeys
- 4. Load testing — k6 on login and list endpoints
- 5. Mobile testing — Capacitor device tests on Android/iOS

Document Version: 1.0
Last Updated: 2026-10-06

08 Deployment

MACVEL School Management System

1. Environments

Environment	Frontend	Backend	Database
Development	Vite dev server (<code>npm run dev</code> , port 5173)	<code>npm run dev</code> (nodemon, port 3000)	Local MongoDB <code>macvel_school</code>
Production	Vercel (static build)	Node.js host (any Node 18+ server)	MongoDB Atlas

2. Frontend Deployment (Vercel)

Setting	Value
Project root	<code>frontend/</code>
Build command	<code>npm run build</code>
Output directory	<code>dist</code> (Vite)
SPA routing	<code>frontend/vercel.json</code> rewrites all non-asset paths → <code>/index.html</code>
API URL	<code>EXPO_PUBLIC_API_URL</code> / <code>.env</code> → production API base URL

Steps:

1. Import the Git repository into Vercel (or `vercel` CLI)
2. Set root directory to `frontend`
3. Add environment variables (`.env.example` as reference)
4. Deploy — every push to `main` triggers an automatic redeploy
5. Verify deep links (`/admin/students` , `/login`) do not 404 on refresh

3. Backend Deployment

Requirements: Node.js ≥ 18, MongoDB (local or Atlas), optional Cloudinary account

```
cd backend
npm ci           # install dependencies
cp .env.example .env # then edit values (see §5)
npm run seed:superadmin # optional: create first Super Admin
npm start        # node src/server.js → port 3000
```

Post-deploy verification:

Check	Command / URL	Expected
Health	<code>GET /health</code>	<code>{ status: "OK", version: "3.0.0" }</code>
DB	Server log	<code>[DB] MongoDB connected successfully</code>

Check	Command / URL	Expected
CORS	Browser call from frontend origin	<code>Access-Control-Allow-Origin</code> present
Uploads	<code>POST /api/files/upload</code>	Secure file URL returned

Operational notes:

- Server binds `0.0.0.0` — put a reverse proxy/HTTPS terminator in front for production
- Failed DB connection at boot exits the process (`process.exit(1)`) — process managers (PM2/systemd) will restart it
- Uploaded files persist either in Cloudinary (recommended) or `backend/uploads/` (must be volume-mounted if local)

4. Mobile Deployment (Capacitor)

Project: `mobile/` (appld `com.school.app`, webDir `build`)

```
cd mobile
npm install
npm run build          # Vite production build → build/
npx cap sync android  # copy web assets + plugins into android/
npx cap sync ios       # copy web assets + plugins into ios/
```

Platform	Tool	Output
Android	Android Studio → *Build > Build APK(s)* / <code>gradlew assembleRelease</code>	<code>app-release.apk</code> / AAB for Play Store
iOS	Xcode → *Product > Archive* (Apple Developer account required)	IPA via Organizer

Set the backend API URL in `mobile/.env` before building (point to production API, not `localhost`).

5. Environment Variables

Variable	Example	Description
<code>PORT</code>	<code>3000</code>	API server port
<code>NODE_ENV</code>	<code>development</code> / <code>production</code>	Environment mode
<code>MONGODB_URI</code>	<code>mongodb+srv://user:pass@cluster/...</code>	MongoDB connection string
<code>JWT_SECRET</code>	<code><strong random></code>	JWT signing key (change in production)
<code>JWT_EXPIRES_IN</code>	<code>7d</code>	Token lifetime
<code>BCRYPT_SALT_ROUNDS</code>	<code>10</code>	Password hashing cost
<code>MAX_FILE_SIZE</code>	<code>10485760</code>	Upload limit (10 MB)
<code>UPLOAD_PATH</code>	<code>./uploads</code>	Local upload folder
<code>CLOUDINARY_URL</code>	<code>cloudinary://key:secret@cloud</code>	Cloudinary storage (empty → local fallback)

Variable	Example	Description
ALLOWED_ORIGINS	https://your-app.vercel.app	Frontend origins
SMTP_HOST / SMTP_USER / SMTP_PASS	—	Optional mail notifications

6. Production Checklist

- ☐ `JWT_SECRET` replaced with a strong random value
- ☐ `NODE_ENV=production`
- ☐ MongoDB Atlas IP allow-list configured; strong DB credentials
- ☐ `CLOUDINARY_URL` set (avoids ephemeral-disk file loss on hosts like Vercel/Render)
- ☐ CORS `ALLOWED_ORIGINS` restricted to the deployed frontend domain
- ☐ HTTPS enabled at the proxy/platform level
- ☐ Super Admin seeded and default passwords changed
- ☐ `GET /health` monitored for uptime
- ☐ Database backups enabled (Atlas snapshot schedule)

Document Version: 1.0

Last Updated: 2026-10-06

09 Development Log

MACVEL School Management System

1. Project Timeline

Item	Value
Repository	<code>felix-fr7/school</code> (branch <code>main</code>)
First commit	2026-06-28 — "first commit"
Latest commit	2026-09-30 — "finish" (<code>574dd4a</code>)
Total commits	80 commits over ~3 months
Commit style	Short milestone messages (<code>finish</code> , <code>fix: ...</code> , <code>feat: ...</code>)

2. Phase-wise Development Log

Phase 1 — Foundation & Multi-Tenant Core (2026-06-28 → 2026-07-04, 10 commits)

Date	Commit	Activity
2026-06-28	<code>first commit</code>	Repository initialisation
2026-06-28	<code>Add complete multi-tenant school management system</code>	Backend (Express + MongoDB), RBAC, tenant isolation, initial frontend
2026-06-29	<code>new dash</code>	Admin dashboard
2026-06-30	<code>new add student</code>	Student creation flow
2026-07-01	<code>new one of student</code>	Student module refinements
2026-07-03 → 07-04	<code>new one , full project</code>	End-to-end wiring of core modules

Phase 2 — Classes, Homework & Ionic Move (2026-07-05 → 2026-07-17, 14 commits)

Date	Activity
2026-07-05 → 07-06	Class management, office-side screens
2026-07-08 → 07-10	New class flows, iteration on dashboards
2026-07-12 → 07-13	Homework module built and finished
2026-07-16	Ionic migration (<code>ionic</code> commit) — React + Ionic UI adopted
2026-07-17	Iteration (<code>newone</code>)

Phase 3 — Super Admin & Admin Portal (2026-07-18 → 2026-08-02, 17 commits)

Date	Activity
2026-07-18 → 07-19	Super admin build + fixes
2026-07-21 → 07-23	New login flow; super admin finished

Date	Activity
2026-07-25	Admin portal finished
2026-07-30 → 07-31	Project restructuring; final super admin
2026-08-01 → 08-02	Admin base/off iteration, new admin features

Phase 4 — Student App & Mobile (2026-08-07 → 2026-08-19, 11 commits)

Date	Activity
2026-08-07 → 08-08	Student module, mobile (Capacitor) project
2026-08-09	Student app + 3 key fixes: Title-Case role comparisons, <code>decoded.userId</code> auth, profile <code>schoolId</code>
2026-08-10	New iteration
2026-08-17 → 08-19	Student entry flow, "student last stage"

Phase 5 — Login Rework & First Deployment (2026-08-22 → 2026-08-26, 9 commits)

Date	Activity
2026-08-22 → 08-24	New login, general finish passes
2026-08-26	Vercel deployment: <code>vercel</code> commit, <code>vercel.json</code> added, SPA rewrites fixed, file moved to <code>frontend/</code> , redeploy triggered

Phase 6 — Stabilisation, Fixes & Storage (2026-09-02 → 2026-09-30, 19 commits)

Date	Activity
2026-09-02 → 09-05	Edit-flow fixes, finish pass
2026-09-08	3 fixes: admin delete/edit unpublished albums/videos/homework (404s); homework feature removed from admin side; 2-bug fix
2026-09-14 → 09-21	Final testing rounds (<code>final test</code> ×2, <code>new finish</code>)
2026-09-24 → 09-25	Album fix, new method, verification pass
2026-09-27 → 09-29	New login polish; esbuild install-script allowance for CI build
2026-09-30	cloudinary add (media storage pipeline) + final <code>finish</code> → v1.0 release

3. Milestone Summary

#	Milestone	Date Reached
1	Multi-tenant backend foundation	2026-06-28
2	Admin dashboard + student management	2026-06-30
3	Homework module complete	2026-07-13
4	Ionic UI migration	2026-07-16

#	Milestone	Date Reached
5	Super Admin portal complete	2026-07-31
6	School Admin portal complete	2026-07-25
7	Student app + mobile (Capacitor)	2026-08-08
8	Production web deployment (Vercel)	2026-08-26
9	Admin content bug-fix batch	2026-09-08
10	Cloudinary storage integration	2026-09-30
11	v1.0.0 Release	2026-09-30

4. Repository Structure Evolved As

```
School/  
├─ backend/      # Express + Mongoose API (src: config, controllers, routes, models, middleware)  
├─ frontend/     # React + Ionic + Vite web app (screens: admin, teacher, superadmin,  
classcontroller)  
├─ mobile/       # Capacitor wrapper + student screens (android/, ios/, student/)  
├─ docs/         # This documentation set (01-10) + generated PDF  
└─ *.js          # Standalone API test scripts
```

Document Version: 1.0
Last Updated: 2026-10-06

10 Release Notes

MACVEL School Management System

Current Release — v1.0.0 (2026-09-30)

Item	Value
Release version	1.0.0 (first stable release)
Release date	2026-09-30
Commit	574dd4a — "finish" (on main)
Backend package	macvel-school-backend v1.0.0 (health API reports version: 3.0.0)
Frontend package	school-ionic-frontend v2.0.0
Mobile package	school-ionic-mobile v1.0.0 (Capacitor, com.school.app)
Node.js	≥ 18 required

1. Highlights

- **Multi-tenant SaaS** — unlimited schools with complete data isolation
- **Five role portals** — Super Admin, School Admin, Teacher, Student, Class Monitor
- **Triple login** — email, student ID, or class code + password
- **Academics suite** — homework (assign → submit → grade), exams + schedules, marks, term report cards (manual or uploaded PDF/image)
- **Weekly lesson diary** — teacher plans and student by-date views
- **Timetable grid** — subject × Mon–Sat classwork/homework with publish workflow
- **Content hub** — news, circulars, gallery, albums (with external links), videos, messages, posts
- **Cloudinary media storage** with automatic local-disk fallback
- **White-label branding** — web + mobile logo and headings, public before login

2. Added in v1.0.0

Area	Features
Auth	Register, login (email/ID), class login, JWT sessions, password change, logout
Super Admin	School CRUD, status/plan control, admin management, stats, cleanup tools
Admin	Classes (auto codes, password reset), students (next-ID), teachers, subjects, contacts, exams, report cards, albums, calendar
Teacher	Dashboard, homework CRUD + grading, submissions view, circulars
Student	Dashboard, homework submission, exams, marks, timetable, profile, report cards, news, weekly lessons
Class monitor	Student bulk import (CSV/XLSX template), homework, exams, subjects, timetable authoring
Files	Multer uploads → Cloudinary (fallback ./uploads), exam/report/homework upload endpoints
Deployment	Vercel SPA hosting (vercel.json rewrites), Capacitor Android/iOS shells

3. Fixed in v1.0.0

Date	Fix
2026-08-09	Content controller role comparisons now Title-Case (<code>Student</code> , <code>Teacher</code>)
2026-08-09	Content routes use <code>decoded.userId</code> and correct Class field names
2026-08-09	Profile endpoint uses <code>schoolId</code> instead of non-existent <code>tenantId</code>
2026-08-26	Vercel deep-link 404s resolved via SPA rewrites in <code>frontend/vercel.json</code>
2026-09-08	Admin delete/edit of unpublished albums, videos and homework no longer 404s
2026-09-08	Homework feature removed from admin side (per business rule)
2026-09-25	Album edit/delete behaviour for admins
2026-09-29	<code>esbuild</code> install script allowed for CI builds

4. Known Limitations

1. **Automated tests** — no unit/E2E framework yet; testing is via manual API scripts (see 07 Testing)
2. **Homework on admin side** — intentionally hidden; admins manage homework indirectly
3. **Products module** — legacy/demo, not part of the school workflow
4. **Unmapped legacy route files** — `backend/src/routes/superadmin.js` and `utils.js` exist but are not mounted in `server.js`
5. **Offline mode** — no offline data caching in the mobile app yet
6. **Email/SMTTP** — configuration present but notifications are not wired to a workflow

5. Upgrade / Installation Notes

```
# Fresh install
git clone https://github.com/felix-fr7/school.git
cd school/backend && npm install && cp .env.example .env # set MONGODB_URI, JWT_SECRET
npm run seed:superadmin && npm start
cd ../frontend && npm install && npm run dev # local web app
cd ../mobile && npm install && npm run build && npx cap sync android
```

- Setting `CLOUDINARY_URL` is recommended before handling real uploads in production
- Changing `JWT_SECRET` invalidates existing tokens — users must log in again

6. Upcoming (Planned Next)

- Automated test suite (Jest + Supertest) and CI pipeline
- Attendance module end-to-end (screen exists; API to be added)
- Push notifications for news/circulars in the mobile app
- Parent role portal and fee management
- Analytics dashboards with trend charts

Document Version: 1.0

Last Updated: 2026-10-06